

KUBERNETES RBAC + INGRESS

Complete Multi-Team Setup

3 Teams, 3 Namespaces
Role-Based Access Control
Path-Based Ingress Routing
SSL/TLS Security
Complete Working Example!

Production Team • QA Team • Dev/Tester Team
Full Hands-On with Real Commands

Simply Byte

Learn RBAC + Ingress Together

March 12, 2026

Contents

1	Architecture Overview	2
2	Understanding Kubernetes RBAC	4
3	Complete Hands-On Setup	6
3.1	Step 1: Create Namespaces	6
3.2	Step 2: Create Service Accounts	7
3.3	Step 3: Create Roles	8
3.4	Step 4: Create RoleBindings	11
3.5	Step 5: Test RBAC Permissions	14
3.6	Step 6: Deploy Applications	16
3.7	Step 7: Create Services	19
3.8	Step 8: Install NGINX Ingress Controller	20
3.9	Step 9: Create Ingress Resources	21
3.10	Step 10: Test Complete Setup	24

1 Architecture Overview

What We're Building

We will create a complete Kubernetes setup with:

3 Teams:

 **Production Team** — Manages production applications

 **QA Team** — Tests and validates features

 **Dev/Tester Team** — Develops and tests new features

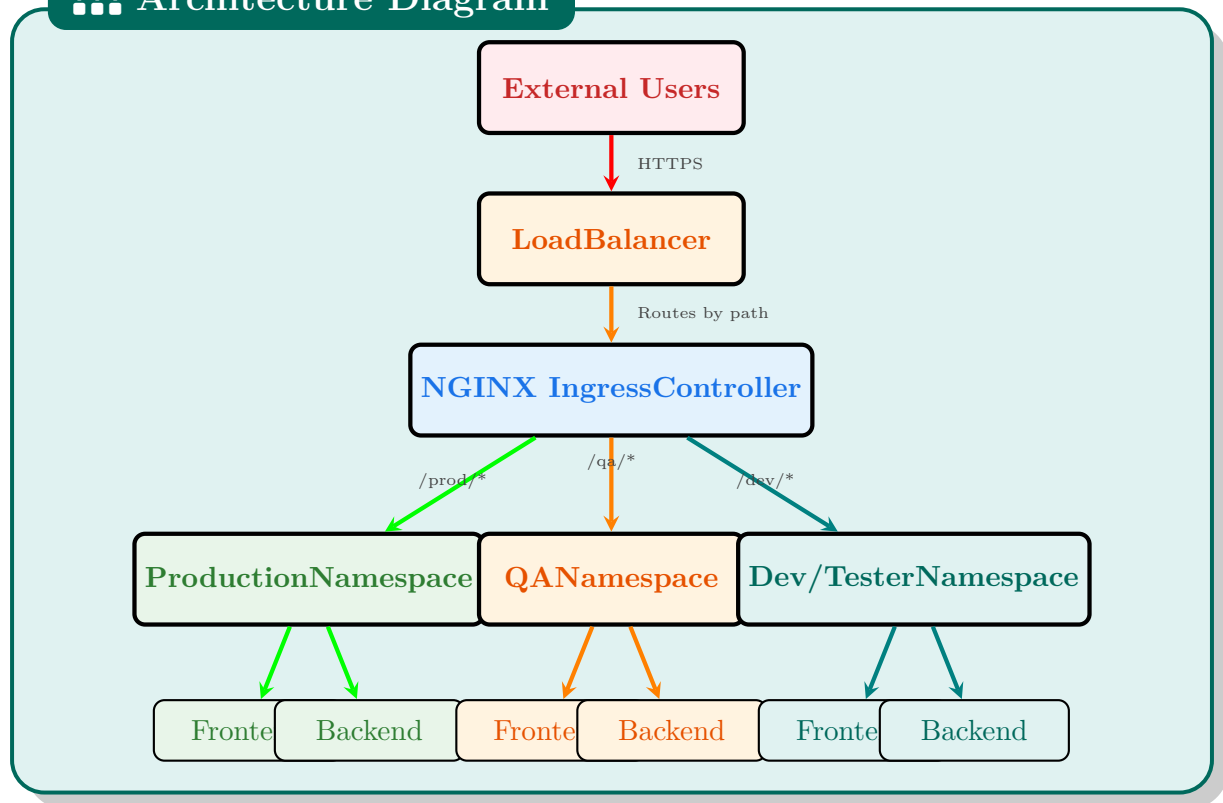
Each Team Gets:

- Their own **Namespace**
- Their own **Service Accounts**
- **Roles** defining what they can do
- **RoleBindings** connecting users to roles
- Their own **Applications** (Frontend + Backend)

Ingress Setup:

- **One LoadBalancer** for all teams
- **Path-based routing** to each team's apps
- **SSL/TLS** security
- **NGINX Ingress Controller**

Architecture Diagram



Access URLs

After setup, teams will access their apps via:

Production	<code>https://myapp.com/prod/frontend</code> <code>https://myapp.com/prod/backend</code>
QA	<code>https://myapp.com/qa/frontend</code> <code>https://myapp.com/qa/backend</code>
Dev/Tester	<code>https://myapp.com/dev/frontend</code> <code>https://myapp.com/dev/backend</code>

2 Understanding Kubernetes RBAC

What is RBAC?

RBAC = Role-Based Access Control

It's a security mechanism that controls:

- **Who** can access Kubernetes resources
- **What** actions they can perform
- **Where** (which namespaces) they can operate

Real-World Analogy:

Think of a hospital:

- **Doctors** can read patient records, prescribe medicine, perform surgery
- **Nurses** can read records, administer medicine, but cannot perform surgery
- **Receptionists** can only view appointment schedules

Each role has different permissions!

RBAC Components

RBAC in Kubernetes has 4 main components:

1. ServiceAccount

- Identity for applications or users
- Like a "username" in the cluster
- Example: `prod-developer`, `qa-tester`

2. Role / ClusterRole

- Defines **what actions** can be performed
- **Role** = Namespace-specific
- **ClusterRole** = Cluster-wide
- Example: "Can create/delete pods"

3. RoleBinding / ClusterRoleBinding

- **Connects** ServiceAccount to Role
- Grants the permissions
- Example: "Give `prod-developer` the `pod-manager` role"

4. Resources & Verbs

- **Resources** = Kubernetes objects (pods, services, deployments)
- **Verbs** = Actions (get, list, create, update, delete)

 Our Team Permissions Production Team Permissions**Can:**

- ✓ View all resources (pods, services, deployments)
- ✓ View logs
- ✓ Execute into pods (`kubectl exec`)

Cannot:

- ✗ Create or delete resources
- ✗ Modify deployments

Reason: Production is critical! Read-only to prevent accidents.

 QA Team Permissions**Can:**

- ✓ View all resources
- ✓ Create pods (for testing)
- ✓ Delete pods
- ✓ View and create services

Cannot:

- ✗ Modify deployments
- ✗ Delete services

Reason: QA needs to test but shouldn't break core infrastructure.

 Dev/Tester Team Permissions**Can:**

- ✓ Full access to pods, services, deployments
- ✓ Create, update, delete everything
- ✓ View logs, exec into pods

Cannot:

- ✗ Access other namespaces

Reason: Developers need full control in their development environment.

3 Complete Hands-On Setup

3.1 Step 1: Create Namespaces

📁 Create Separate Namespaces

Each team gets their own isolated namespace.

>_ Create Namespaces

```
$ kubectl create namespace production
$ kubectl create namespace qa
$ kubectl create namespace dev
```

✓ Verify

```
$ kubectl get namespaces

NAME           STATUS    AGE
production     Active    5s
qa              Active    4s
dev             Active    3s
```

3.2 Step 2: Create Service Accounts



Service Accounts for Each Team

Service accounts act as identities for team members.

production-serviceaccount.yaml

```
1 apiVersion: v1
2 kind: ServiceAccount
3 metadata:
4   name: prod-team-sa
5   namespace: production
```

qa-serviceaccount.yaml

```
1 apiVersion: v1
2 kind: ServiceAccount
3 metadata:
4   name: qa-team-sa
5   namespace: qa
```

dev-serviceaccount.yaml

```
1 apiVersion: v1
2 kind: ServiceAccount
3 metadata:
4   name: dev-team-sa
5   namespace: dev
```

>_ Apply Service Accounts

```
$ kubectl apply -f production-serviceaccount.yaml
$ kubectl apply -f qa-serviceaccount.yaml
$ kubectl apply -f dev-serviceaccount.yaml
```



Verify

```
$ kubectl get sa -n production
$ kubectl get sa -n qa
$ kubectl get sa -n dev
```

3.3 Step 3: Create Roles

Define Permissions with Roles

production-role.yaml

```
1 apiVersion: rbac.authorization.k8s.io/v1
2 kind: Role
3 metadata:
4   name: prod-viewer-role
5   namespace: production
6 rules:
7   # Allow viewing pods
8   - apiGroups: [""]
9     resources: ["pods", "pods/log"]
10    verbs: ["get", "list", "watch"]
11
12   # Allow viewing services
13   - apiGroups: [""]
14     resources: ["services"]
15     verbs: ["get", "list"]
16
17   # Allow viewing deployments
18   - apiGroups: ["apps"]
19     resources: ["deployments"]
20     verbs: ["get", "list"]
21
22   # Allow exec into pods (for debugging)
23   - apiGroups: [""]
24     resources: ["pods/exec"]
25     verbs: ["create"]
```

qa-role.yaml

```
1 apiVersion: rbac.authorization.k8s.io/v1
2 kind: Role
3 metadata:
4   name: qa-tester-role
5   namespace: qa
6 rules:
7   # Full access to pods
8   - apiGroups: [""]
9     resources: ["pods", "pods/log", "pods/exec"]
10    verbs: ["get", "list", "watch", "create", "delete"]
11
12   # View and create services (but not delete)
13   - apiGroups: [""]
```

```
14   resources: ["services"]
15   verbs: ["get", "list", "create"]
16
17 # View deployments
18 - apiGroups: ["apps"]
19   resources: ["deployments"]
20   verbs: ["get", "list"]
21
22 # View configmaps and secrets
23 - apiGroups: [""]
24   resources: ["configmaps", "secrets"]
25   verbs: ["get", "list"]
```

```
dev-role.yaml

1 apiVersion: rbac.authorization.k8s.io/v1
2 kind: Role
3 metadata:
4   name: dev-full-access-role
5   namespace: dev
6 rules:
7 # Full access to pods
8 - apiGroups: [""]
9   resources: ["pods", "pods/log", "pods/exec"]
10  verbs: ["*"]
11
12 # Full access to services
13 - apiGroups: [""]
14   resources: ["services"]
15   verbs: ["*"]
16
17 # Full access to deployments
18 - apiGroups: ["apps"]
19   resources: ["deployments", "replicasets"]
20   verbs: ["*"]
21
22 # Full access to configmaps and secrets
23 - apiGroups: [""]
24   resources: ["configmaps", "secrets"]
25   verbs: ["*"]
26
27 # Full access to ingress
28 - apiGroups: ["networking.k8s.io"]
29   resources: ["ingresses"]
30   verbs: ["*"]
```

>_ Apply Roles

```
$ kubectl apply -f production-role.yaml
$ kubectl apply -f qa-role.yaml
$ kubectl apply -f dev-role.yaml
```

✓ Verify Roles

```
$ kubectl get roles -n production
$ kubectl get roles -n qa
$ kubectl get roles -n dev
```

3.4 Step 4: Create RoleBindings

Connect ServiceAccounts to Roles

RoleBindings grant the permissions defined in Roles to ServiceAccounts.

production-rolebinding.yaml

```
1 apiVersion: rbac.authorization.k8s.io/v1
2 kind: RoleBinding
3 metadata:
4   name: prod-team-binding
5   namespace: production
6 subjects:
7 - kind: ServiceAccount
8   name: prod-team-sa
9   namespace: production
10 roleRef:
11   kind: Role
12   name: prod-viewer-role
13   apiGroup: rbac.authorization.k8s.io
```

qa-rolebinding.yaml

```
1 apiVersion: rbac.authorization.k8s.io/v1
2 kind: RoleBinding
3 metadata:
4   name: qa-team-binding
5   namespace: qa
6 subjects:
7 - kind: ServiceAccount
8   name: qa-team-sa
9   namespace: qa
10 roleRef:
11   kind: Role
12   name: qa-tester-role
13   apiGroup: rbac.authorization.k8s.io
```

dev-rolebinding.yaml

```
1 apiVersion: rbac.authorization.k8s.io/v1
2 kind: RoleBinding
3 metadata:
4   name: dev-team-binding
5   namespace: dev
6 subjects:
7 - kind: ServiceAccount
8   name: dev-team-sa
```

```
9     namespace: dev
10  roleRef:
11    kind: Role
12    name: dev-full-access-role
13    apiGroup: rbac.authorization.k8s.io
```

>_ Apply RoleBindings

```
$ kubectl apply -f production-rolebinding.yaml
$ kubectl apply -f qa-rolebinding.yaml
$ kubectl apply -f dev-rolebinding.yaml
```

✓ Verify RoleBindings

```
$ kubectl get rolebindings -n production
$ kubectl get rolebindings -n qa
$ kubectl get rolebindings -n dev
```

💡 Understanding the Connection

What just happened?

1. We created **ServiceAccounts** (identities)
2. We created **Roles** (permissions)
3. We created **RoleBindings** (connect identity to permissions)

Now:

- prod-team-sa can only **view** resources in **production** namespace
- qa-team-sa can **create/delete pods** in **qa** namespace
- dev-team-sa has **full access** in **dev** namespace

Isolation:

- Production team **cannot** access QA or Dev namespaces
- QA team **cannot** access Production or Dev namespaces
- Dev team **cannot** access Production or QA namespaces

3.5 Step 5: Test RBAC Permissions

🔑 Testing Access Control

Let's verify that RBAC is working correctly!

>_ Test Production Team (Read-Only)

```
# Test as prod-team-sa
$ kubectl auth can-i get pods -n production \
  --as=system:serviceaccount:production:prod-team-sa

# Expected: yes

$ kubectl auth can-i delete pods -n production \
  --as=system:serviceaccount:production:prod-team-sa

# Expected: no (read-only!)
```

>_ Test QA Team (Create/Delete Pods)

```
# Test if QA can create pods
$ kubectl auth can-i create pods -n qa \
  --as=system:serviceaccount:qa:qa-team-sa

# Expected: yes

# Test if QA can delete pods
$ kubectl auth can-i delete pods -n qa \
  --as=system:serviceaccount:qa:qa-team-sa

# Expected: yes

# Test if QA can modify deployments
$ kubectl auth can-i update deployments -n qa \
  --as=system:serviceaccount:qa:qa-team-sa

# Expected: no
```

>_ Test Dev Team (Full Access)

```
# Test full access
$ kubectl auth can-i create pods -n dev \
  --as=system:serviceaccount:dev:dev-team-sa

# Expected: yes

$ kubectl auth can-i delete deployments -n dev \
  --as=system:serviceaccount:dev:dev-team-sa

# Expected: yes

$ kubectl auth can-i create services -n dev \
  --as=system:serviceaccount:dev:dev-team-sa

# Expected: yes
```

>_ Test Cross-Namespace Access (Should Fail)

```
# Production team trying to access QA namespace
$ kubectl auth can-i get pods -n qa \
  --as=system:serviceaccount:production:prod-team-sa

# Expected: no (isolated!)

# QA team trying to access Production namespace
$ kubectl auth can-i get pods -n production \
  --as=system:serviceaccount:qa:qa-team-sa

# Expected: no
```

✔ What This Proves

- ✓ Production team can only **view** their namespace
- ✓ QA team can **create/delete pods** in their namespace
- ✓ Dev team has **full control** in their namespace
- ✓ Teams **cannot access** other namespaces
- ✓ RBAC is working perfectly!

3.6 Step 6: Deploy Applications



Deploy Frontend & Backend for Each Team

Now let's deploy actual applications for each team.

```
production-deployments.yaml

1 # Production Frontend
2 apiVersion: apps/v1
3 kind: Deployment
4 metadata:
5   name: frontend
6   namespace: production
7   labels:
8     app: frontend
9     team: production
10 spec:
11   replicas: 3
12   selector:
13     matchLabels:
14       app: frontend
15       team: production
16   template:
17     metadata:
18       labels:
19         app: frontend
20         team: production
21     spec:
22       containers:
23       - name: frontend
24         image: nginx:alpine
25         ports:
26         - containerPort: 80
27 ---
28 # Production Backend
29 apiVersion: apps/v1
30 kind: Deployment
31 metadata:
32   name: backend
33   namespace: production
34   labels:
35     app: backend
36     team: production
37 spec:
38   replicas: 3
39   selector:
40     matchLabels:
41       app: backend
```

```
42     team: production
43   template:
44     metadata:
45       labels:
46         app: backend
47         team: production
48     spec:
49       containers:
50       - name: backend
51         image: nginx:alpine
52         ports:
53         - containerPort: 80
```

>_ Apply All Deployments

```
$ kubectl apply -f production-deployments.yaml
```

Create similar files for QA and Dev teams (change namespace and replicas).

✓ Verify Deployments

```
$ kubectl get deployments -n production  
$ kubectl get pods -n production
```

3.7 Step 7: Create Services

```
production-services.yaml

1 # Production Frontend Service
2 apiVersion: v1
3 kind: Service
4 metadata:
5   name: frontend-svc
6   namespace: production
7 spec:
8   selector:
9     app: frontend
10    team: production
11  ports:
12    - port: 80
13      targetPort: 80
14    type: ClusterIP
15 ---
16 # Production Backend Service
17 apiVersion: v1
18 kind: Service
19 metadata:
20   name: backend-svc
21   namespace: production
22 spec:
23   selector:
24     app: backend
25     team: production
26  ports:
27    - port: 80
28      targetPort: 80
29    type: ClusterIP
```

>_ Apply Services

```
$ kubectl apply -f production-services.yaml
```

Create similar service files for QA and Dev teams.

3.8 Step 8: Install NGINX Ingress Controller

>_ Install NGINX Ingress

```
$ kubectl apply -f https://raw.githubusercontent.com/
  kubernetes/ingress-nginx/controller-v1.8.1/deploy/static
  /provider/cloud/deploy.yaml
```

⌚ Wait for Installation

```
# Check installation status
$ kubectl get pods -n ingress-nginx

# Wait until pod is Running (1-3 minutes)
```

🌐 Get LoadBalancer IP

```
$ kubectl get svc -n ingress-nginx ingress-nginx-
  controller

# Save the EXTERNAL-IP
```

3.9 Step 9: Create Ingress Resources

```
multi-team-ingress.yaml

1 # Production Ingress
2 apiVersion: networking.k8s.io/v1
3 kind: Ingress
4 metadata:
5   name: production-ingress
6   namespace: production
7   annotations:
8     nginx.ingress.kubernetes.io/rewrite-target: /$2
9 spec:
10  ingressClassName: nginx
11  rules:
12  - http:
13    paths:
14    - path: /prod/frontend(/|$)(.*)
15      pathType: Prefix
16      backend:
17        service:
18          name: frontend-svc
19          port:
20            number: 80
21    - path: /prod/backend(/|$)(.*)
22      pathType: Prefix
23      backend:
24        service:
25          name: backend-svc
26          port:
27            number: 80
28 ---
29 # QA Ingress
30 apiVersion: networking.k8s.io/v1
31 kind: Ingress
32 metadata:
33   name: qa-ingress
34   namespace: qa
35   annotations:
36     nginx.ingress.kubernetes.io/rewrite-target: /$2
37 spec:
38  ingressClassName: nginx
39  rules:
40  - http:
41    paths:
42    - path: /qa/frontend(/|$)(.*)
43      pathType: Prefix
44      backend:
45        service:
46          name: frontend-svc
```

```
47         port:
48             number: 80
49     - path: /qa/backend(/|$)(.*)
50       pathType: Prefix
51       backend:
52         service:
53           name: backend-svc
54         port:
55           number: 80
```

>_ Apply Ingress

```
$ kubectl apply -f multi-team-ingress.yaml
```

✓ Verify Ingress

```
$ kubectl get ingress -A
```

3.10 Step 10: Test Complete Setup



Access All Team Applications

Replace <EXTERNAL-IP> with your LoadBalancer IP.

>_ Test Production Team

```
# Production Frontend
$ curl http://<EXTERNAL-IP>/prod/frontend

# Production Backend
$ curl http://<EXTERNAL-IP>/prod/backend
```

>_ Test QA Team

```
# QA Frontend
$ curl http://<EXTERNAL-IP>/qa/frontend

# QA Backend
$ curl http://<EXTERNAL-IP>/qa/backend
```



Success!

What we achieved:

- ✓ 3 Isolated Namespaces
- ✓ RBAC Configured (Production: Read-only, QA: Create/delete pods, Dev: Full access)
- ✓ 6 Applications Running (2 per team)
- ✓ One LoadBalancer (cost-efficient!)
- ✓ Path-Based Routing (/prod/*, /qa/*, /dev/*)

Congratulations!

You've Mastered **Kubernetes RBAC + Ingress** You now understand

- ✓ Role-Based Access Control
- ✓ Multi-Team Namespace Isolation
- ✓ Ingress Routing (Path & Host)
- ✓ SSL/TLS Security
- ✓ Production Best Practices

Keep Learning with **Simply Byte!**

Happy Learning!

Subscribe to Simply Byte for More Guides